

Arquitectura de Microservicios con Contenedores

Docentes

- Ing. Federico Castañeda
- Ing. Carlos Maceira
- Ing. Franco Rupnik

Usando Docker

CLI • redes • volúmenes

Objetivos de la clase

Al finalizar esta clase vas a poder:

- **Manejar** el ciclo de vida de un container desde la CLI y diagnosticarlo cuando falla
- **Explicar** qué hace el engine y qué hace el kernel en cada modo de red
- **Elegir** el driver de red correcto y publicar puertos entendiendo el NAT que hay detrás
- **Distintuir** volumes, bind mounts y tmpfs, y saber dónde vive cada dato
- **Anticipar** los problemas clásicos de permisos, persistencia y DNS

Arquitectura cliente-servidor

docker es solo un **cliente HTTP**. No crea containers:
le pide al **daemon** que los cree.

El daemon (dockerd) expone una **REST API** y
delega en containerd → runc → kernel.

Consecuencia práctica: docker ps en tu laptop
puede listar containers de otra máquina.

docker (CLI)

cliente HTTP

unix:///var/run/docker.sock

REST API

dockerd

imágenes, redes, volúmenes

containerd + runc

bundle OCI, clone(), cgroups

Linux kernel

namespaces · cgroups · overlay

El socket es la superficie de ataque

la API sin la CLI

```
curl --unix-socket /var/run/docker.sock http://localhost/containers/json  
DOCKER_HOST=ssh://user@servidor docker ps      # mismo cliente, otro daemon
```

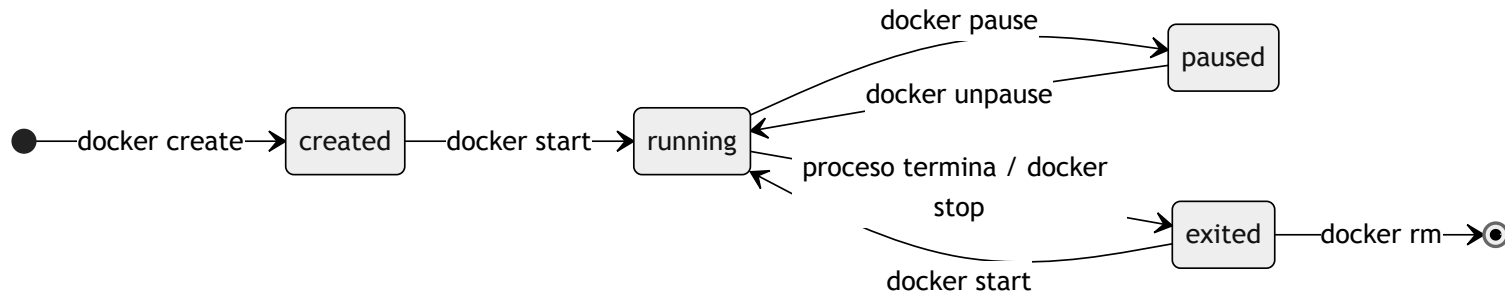
Hecho	Consecuencia
El socket lo posee root:docker	Estar en el grupo docker equivale a ser root
La API permite montar / del host	Un container con el socket montado puede tomar la máquina
No hay autenticación en el socket	La autorización es el permiso del archivo

La CLI: objeto + verbo

Estructura moderna: `docker <objeto> <verbo>`. Los comandos "cortos" son alias históricos.

Objeto	Comandos típicos	Alias corto
container	<code>run ls logs exec stop rm inspect</code>	<code>docker run, docker ps</code>
image	<code>pull ls tag push rm history</code>	<code>docker images, docker rmi</code>
network	<code>create ls connect inspect rm</code>	–
volume	<code>create ls inspect rm</code>	–
system	<code>df prune info events</code>	–

Ciclo de vida



`docker run` = `create` + `start`. `docker rm` borra el container y su capa de escritura.

docker pause usa el cgroup freezer: los procesos quedan congelados, no reciben señales.

Anatomía de docker run

```
docker run -d -p 8080:80 -v datos:/data nginx:1.27 nginx -g 'daemon off;'
```

DETACH PUERTO VOLUMEN IMAGEN COMANDO

Todo lo que va **después de la imagen** es el comando: reemplaza al CMD de la imagen.

Sin `--name`, el daemon inventa uno (`nostalgic_turing`) y lo tenés que buscar cada vez.

Flags que vas a usar siempre

Flag	Qué hace	Qué toca por debajo
<code>-d / -it</code>	Background / interactivo con TTY	stdin y pty del proceso
<code>--rm</code>	Borra el container al terminar	capa RW + metadata
<code>-e / --env-file</code>	Variables de entorno	env del <code>exec()</code>
<code>-p / -P</code>	Publica puertos	reglas DNAT de iptables
<code>-v / --mount</code>	Volúmenes y bind mounts	mounts en el mount namespace
<code>--network</code>	Modo de red	network namespace
<code>-u, -w</code>	Usuario y working dir	UID/GID y <code>chdir</code>
<code>--memory, --cpus</code>	Límites	<code>memory.max</code> , <code>cpu.max</code> (cgroup v2)
<code>--restart</code>	Política de reinicio	supervisión del daemon

Qué pasa cuando hacés `docker run`

1. La CLI arma un `POST /containers/create` y lo manda al socket
2. El daemon **resuelve la referencia** de la imagen y hace `pull` si falta
3. `containerd` desempaqueta los layers → `lowerdir`; crea `upperdir` para este container
4. El daemon **crea el endpoint de red**: `netns`, par `veth`, IP del IPAM, reglas de `iptables`
5. Genera el `config.json` OCI: `namespaces`, `mounts`, `cgroups`, `capabilities`
6. `runc` hace `clone()`, aplica `cgroups`, `pivot_root`, `seccomp` y `exec()`

Los pasos 3 a 6 son exactamente los que hicimos a mano en la demo.

Inspección: la caja de herramientas

Comando	Para qué
<code>docker ps -a</code>	Qué corre, qué murió y con qué exit code
<code>docker logs -f --tail 100 api</code>	Stdout/stderr del proceso principal
<code>docker inspect api</code>	JSON completo: mounts, red, config, estado
<code>docker exec -it api sh</code>	Shell adentro del container que ya corre
<code>docker stats</code>	CPU/RAM/red en vivo (lee los cgroups)
<code>docker top api</code>	Procesos, con los PID del host
<code>docker diff api</code>	Qué cambió respecto de la imagen (el upperdir)
<code>docker events</code>	Stream de eventos del daemon

inspect con formato

```
docker inspect -f '{{ .NetworkSettings.IPAddress }}' api
docker inspect -f '{{ json .Mounts }}' api | jq
```

Logs: qué son y dónde están

`docker logs` **no** lee un archivo del container: lee lo que el daemon capturó de **stdout y stderr** del PID 1.

Si tu app escribe a `/var/log/app.log`, `docker logs` no muestra nada.

Por eso la convención en containers es **loguear a stdout**.

Driver	Dónde va
json-file	/var/lib/docker/containers/<id>/*.js (default)
local	Formato binario, más eficiente
journald	journal de systemd
fluentd, gelf, awslogs	Colector externo

```
docker run --log-opt max-size=10m \  
           --log-opt max-file=3 ...
```

exec vs attach

`docker exec` lanza un **proceso nuevo** dentro de los namespaces del container.

Salir del shell no afecta al proceso principal. Es lo que querés el 99% de las veces.

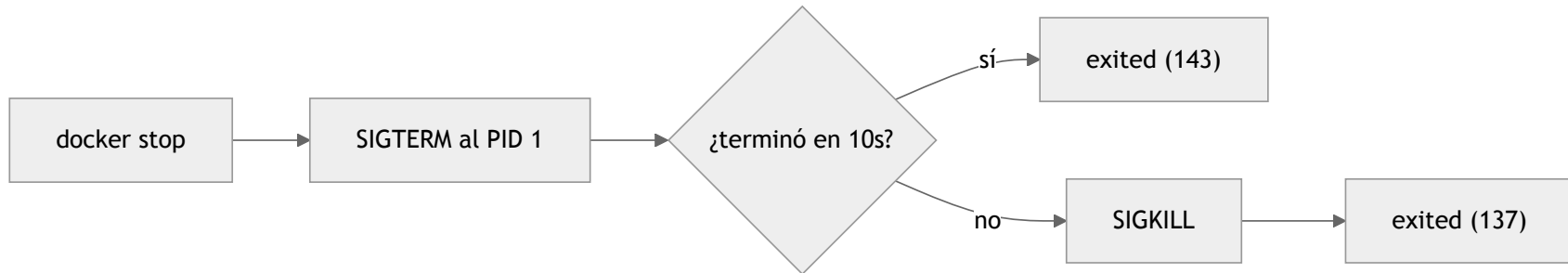
```
docker exec -it api sh
docker exec -u root api apk add curl
```

`docker attach` te conecta al **stdin/stdout del PID 1**.

Ctrl-C le manda SIGINT al proceso principal y **mata el container**.

Para desconectarse sin matarlo: Ctrl-P Ctrl-Q.
-i mantiene stdin abierto · -t asigna un pseudo-TTY

Señales y parada ordenada



- `docker stop -t 30` cambia el período de gracia · `docker kill -s HUP` manda otra señal
- **PID 1 no tiene handlers por defecto:** si tu proceso ignora SIGTERM, siempre esperarás los 10 s
- `--init` inyecta un init mínimo (`tini`) que reapea zombies y reenvía señales

Mover cosas dentro y fuera

Comando	Qué hace	Cuándo
<code>docker cp</code>	Copia archivos host ↔ container	Sacar un dump, meter un config puntual
<code>docker commit</code>	Crea una imagen desde un container	Casi nunca: anti-patrón
<code>docker export / import</code>	Tar del filesystem , sin historia ni layers	Aplanar una imagen
<code>docker save / load</code>	Tar de la imagen con layers y metadata	Mover imágenes sin registry

Imágenes y limpieza

```
docker pull nginx:1.27          docker images
docker tag nginx:1.27 midominio.io/nginx:1.27
docker push midominio.io/nginx:1.27
```

Comando	Qué borra
docker system df	(no borra) muestra cuánto ocupan imágenes, containers, volúmenes y cache
docker container prune	Containers detenidos
docker image prune -a	Imágenes sin container que las use
docker volume prune	Volúmenes sin usar – puede borrar datos
docker system prune -a --volumes	Todo lo anterior

Cómo se construyen y versionan esas imágenes: **clase 04**.

Redes

Qué hace el engine y qué hace el kernel

Punto de partida: el net namespace

Cada container arranca con su **propio network namespace**: interfaces, tabla de rutas, reglas de firewall y puertos propios.

Por eso **dos containers pueden escuchar en el puerto 80** sin chocar.

El "modo de red" define solo una cosa: **cómo se conecta ese namespace al mundo**.

A terminal window with a dark background and light-colored text. The title bar at the top shows three colored circles (red, yellow, green) followed by the text "dentro del container". The terminal content shows the execution of two commands: "ip addr" and "ip route". The output of "ip addr" shows two interfaces: "lo" (loopback) with IP 127.0.0.1 and "eth0@if13" (veth interface) with IP 172.17.0.2. The output of "ip route" shows a default route via 172.17.0.1 on the eth0 interface.

```
dentro del container

$ ip addr
1: lo: <LOOPBACK,UP>
    inet 127.0.0.1/8
12: eth0@if13: <BROADCAST,UP>
    inet 172.17.0.2/16
$ ip route
default via 172.17.0.1 dev eth0
```

eth0@if13: la punta 12 de un par veth cuyo otro extremo es la interfaz 13 del host.

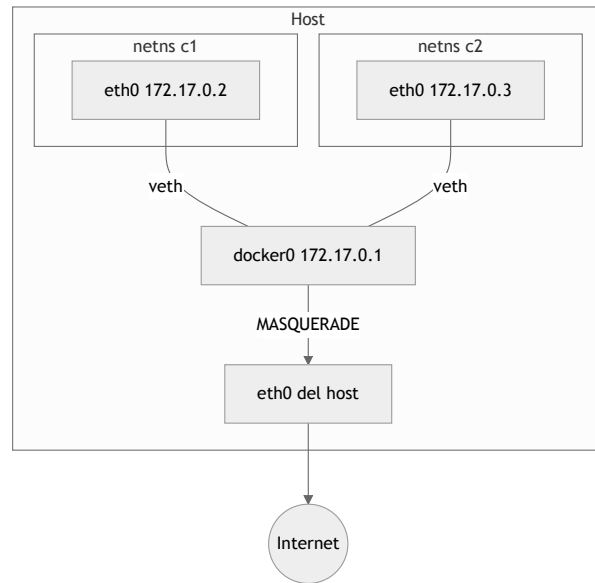
Los modos de red

Driver	Idea	Aislamiento
bridge	Netns propio + switch virtual + NAT (default)	Alto
host	Sin netns propio: usa el del host	Ninguno
none	Netns propio con solo <code>lo</code>	Total
container:<id>	Comparte el netns de otro container	Compartido
macvlan / ipvlan	Interfaz propia sobre la NIC física, IP de la LAN	L2 de la red física
overlay	Red L2 entre varios hosts vía VXLAN	Multi-host

```
docker network ls # bridge, host, none vienen predefinidas
```

Bridge: qué construye el engine

1. Crea un **bridge** (docker0 o br-<id>) – un switch L2 en software
2. Le asigna una subnet del **IPAM** (172.17.0.0/16)
3. Por container: un par **veth**, una punta adentro del netns como eth0, la otra enslavada al bridge
4. Le da al container una IP libre y **ruta por defecto** hacia la IP del bridge
5. Escribe reglas de **iptables** para el egress y para los puertos publicados



Bridge: qué hace el kernel

Objeto del kernel	Quién lo pide	Qué logra
netns	runc vía <code>clone(CLONE_NEWNET)</code>	Stack de red privado
veth pair	Daemon vía netlink	Cable virtual entre netns y host
bridge (br_netfilter)	Driver bridge del kernel	Reenvío L2 entre containers
<code>net.ipv4.ip_forward=1</code>	El daemon lo activa al arrancar	Routing entre bridge y NIC
Reglas nat / filter	Daemon vía iptables/nftables	SNAT de salida y DNAT de entrada

El tráfico container → internet sale **enmascarado con la IP del host** (SNAT). Desde afuera, el container no existe.

Publicar un puerto es hacer DNAT

```
docker run -d -p 8080:80 nginx
```

HOST:CONTAINER

```
# tabla nat, cadena DOCKER (simplificado)
-A DOCKER ! -i docker0 -p tcp --dport 8080 -j DNAT --to-destination 172.17.0.2:80
# y para el egress
-A POSTROUTING -s 172.17.0.0/16 ! -o docker0 -j MASQUERADE
```

- `-p 8080:80` → puerto **8080 del host** hacia el **80 del container**
- `-p 127.0.0.1:8080:80` lo publica **solo en loopback**: no queda expuesto en la LAN
- `-P` publica todos los EXPOSE de la imagen en puertos altos aleatorios

El firewall del host no te protege

Docker inserta sus reglas **antes** que las de ufw o firewalld.

Un `-p 8080:80` queda accesible desde la LAN aunque ufw diga que el 8080 está cerrado.

La cadena **DOCKER-USER** se evalúa antes que las reglas de Docker: es el lugar previsto para tus propias restricciones.

```
iptables -I DOCKER-USER -i eth0 \  
! -s 10.0.0.0/8 -j DROP
```

O más simple: publicar en `127.0.0.1` y poner un reverse proxy adelante.

bridge default vs redes propias

Red bridge por defecto

- Sin resolución DNS por nombre: solo IPs
- Todos los containers en la misma red plana
- `--link` es legacy y está deprecado

Red creada por vos

```
docker network create app-net
docker run -d --name db --network app-net postgres
docker run --rm --network app-net alpine \
  ping -c1 db      # ← resuelve por nombre
```

- **DNS interno** por nombre de container y por alias
- Aislada de otras redes; conectable en caliente

El DNS embebido

En una red propia, el `/etc/resolv.conf` del container apunta a **127.0.0.11**.

Ese resolver **es el daemon**, escuchando dentro del netns del container.

- Nombre de container, alias de red y nombre de servicio → IP interna
- Lo que no resuelve, lo **reenvía** a los DNS del host

```
dentro del container

$ cat /etc/resolv.conf
nameserver 127.0.0.11
options ndots:0

$ getent hosts db
172.19.0.3    db
```

Se puede sobrescribir con `--dns` y agregar entradas fijas con `--add-host`.

host: sin aislamiento de red

```
docker run -d --network host nginx
```

El proceso **no tiene netns propio**: usa el del host.
runc simplemente **no** pasa CLONE_NEWNET.

- Sin veth, sin bridge, **sin NAT**
- `-p` se ignora: la app ya está en las interfaces del host
- Choques de puertos como cualquier proceso

Cuándo sirve

- Throughput alto o latencia crítica (se ahorra el hop del bridge y el NAT)
- Apps que necesitan ver la red real: descubrimiento por broadcast/multicast, monitoreo

Cuándo no

- Multi-tenant o cualquier cosa expuesta: el container ve `localhost` del host, incluidos servicios que creías privados

none y container:

--network none

Netns propio con **solo** lo. Sin veth, sin rutas.

Para jobs puramente de cómputo, o para procesar archivos no confiables sin salida a internet.

```
docker run --network none alpine ip addr
```

--network container:<id>

Comparte el netns de otro container: misma IP, mismas interfaces, mismo localhost.

```
docker run --network container:api netshoot ss -ltnp
```

Es el modelo del **pod de Kubernetes** y de los sidecars: varios procesos, una sola identidad de red.

macvlan e ipvlan

El kernel crea una **sub-interfaz** sobre la NIC física. El container obtiene una **IP de la LAN** y aparece como un equipo más.

- macvlan: **MAC propia** por container → el switch la ve; suele requerir modo promiscuo
- ipvlan: **comparte la MAC** del host, separa por IP (L2) o rutea (L3)

```
docker network create -d macvlan \  
--subnet=192.168.1.0/24 \  
--gateway=192.168.1.1 \  
-o parent=eth0 lan
```

- Sin NAT y sin publicación de puertos: la IP es alcanzable directo
- **El host no se comunica** con sus propios containers macvlan (el kernel no hace hairpin)
- Muchos entornos cloud y Wi-Fi lo bloquean

overlay: red entre varios hosts

Una L2 virtual que abarca **varios nodos**. Es la base de Docker Swarm.

Los paquetes del container se **encapsulan en VXLAN** (UDP 4789) y viajan por la red física entre hosts.

El plano de control (qué IP está en qué nodo) lo distribuye el cluster.



Costo: **MTU menor** (~1450) por el encabezado, y overhead de encapsulado. `--opt encrypted` agrega IPsec.

Cuándo usar cada modo

Modo	Usalo cuando	Evitalo cuando
bridge propio	Es el caso normal: stack local, Compose	–
bridge default	Nunca en algo real	Necesitás DNS o aislamiento
host	Latencia/throughput crítico, escaneo de red	Multi-tenant o exposición pública
none	Jobs sin red, procesamiento no confiable	La app necesita salir
container:	Sidecars, debugging con netshoot	Necesitás aislar
macvlan / ipvlan	Legacy con IP propia en la LAN	Cloud, Wi-Fi, o si el host debe hablarles
overlay	Varios hosts (Swarm)	Un solo host

Comandos de red

```
docker network create --driver bridge --subnet 10.10.0.0/24 app-net
docker network create --internal db-net          # sin salida a internet
docker network ls
docker network inspect app-net                   # containers, IPs, opciones
docker network connect db-net api                # en caliente
docker network disconnect app-net api
docker network prune
```

Un container puede estar en **varias redes a la vez**: una interfaz eth0, eth1... por red.

Patrón típico: api en la red pública **y** en la red interna; db **solo** en la interna.

Diagnóstico de red

receta de debugging

```
docker network inspect app-net | jq '[0].Containers' # ¿están en la misma red?
docker exec api getent hosts db                      # ¿resuelve el nombre?
docker exec api sh -c 'nc -zv db 5432'               # ¿llega al puerto?
docker run --rm --network container:api nicolaka/netshoot ss -ltnp
sudo iptables -t nat -L DOCKER -n                   # ¿está el DNAT?
```

Orden mental: **misma red** → **nombre resuelve** → **puerto abierto** → **la app escucha en 0.0.0.0**.

Volúmenes

Dónde vive el estado

El problema: la capa de escritura

Todo lo que el container escribe va al `upperdir` del overlay. Ese directorio se borra con `docker rm`.

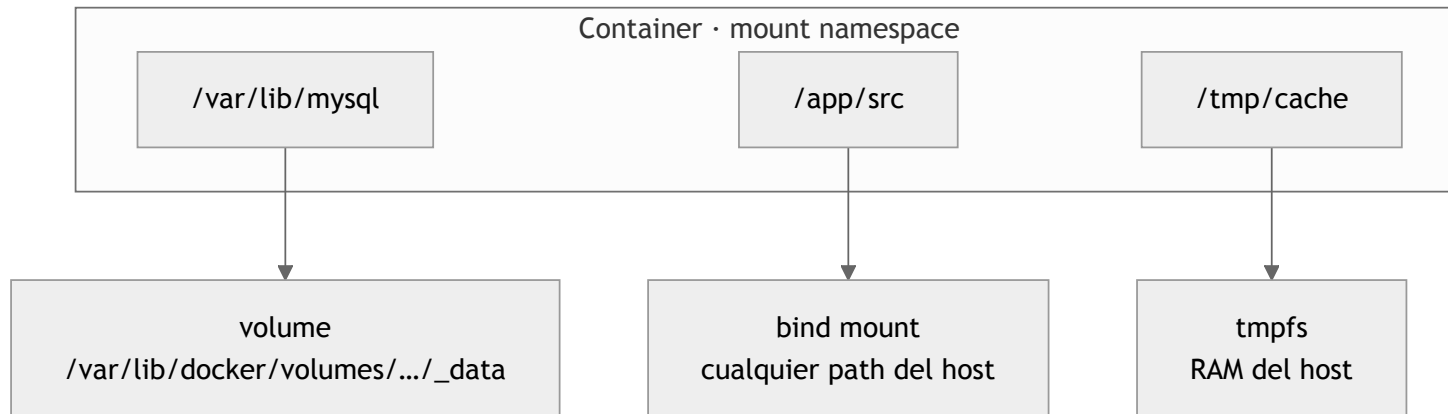
Además, modificar un archivo grande que viene de la imagen dispara un **copy-up**: el kernel copia el archivo entero al `upperdir` antes de escribirlo.

Consecuencias

- Sin volumen, `docker rm` = **datos perdidos**
- Escrituras intensivas sobre la capa RW son **lentas** (copy-up + union)
- Los datos quedan **atados al ciclo de vida** de un container

Bases de datos, uploads y logs **no** van en la capa de escritura.

Los tres tipos de mount



Tipo	Lo gestiona	Sobrevive a rm	Uso típico
volume	Docker	Sí	Datos de producción, bases
bind mount	Vos (path del host)	Sí (es del host)	Código en desarrollo, configs
tmpfs	Kernel (RAM)	No	Secretos, scratch

Dos sintaxis

-v / --volume – corta, posicional

```
-v datos:/var/lib/mysql  
-v /host/app:/app:ro  
-v /app/node_modules      # anónimo
```

Ambigua: si el primero **no** tiene /, es un volumen nombrado; si tiene, es un bind mount.

Y **crea el directorio del host** si no existe.

--mount – explícita, con clave=valor

```
--mount type=volume,src=datos,dst=/var/lib/mysql  
--mount type=bind,src=/host/app,dst=/app,readonly  
--mount type=tmpfs,dst=/tmp,tmpfs-size=64m
```

Falla si el origen del bind no existe: es lo que querés.

Volúmenes nombrados

```
docker volume create datos
docker run -v datos:/var/lib/mysql mysql
```

Viven en
/var/lib/docker/volumes/<nombre>/_data,
fuera de la unión de OverlayFS.

Independientes del container: se pueden compartir,
respaldar y reusar.

Detalles que importan

- Si el volumen está **vacío**, Docker **copia el contenido** que la imagen tenga en ese path (permisos incluidos)
- Un volumen **anónimo** (`-v /path`) recibe un ID aleatorio y queda huérfano al borrar el container
- **VOLUME** en el Dockerfile crea uno anónimo en cada `docker run`

Bind mounts: el kernel al desnudo

Un bind mount es literalmente `mount --bind`: el mismo inodo del host, visible en otro path **dentro del mount namespace** del container.

Opción	Qué hace
<code>readonly / :ro</code>	Monta con <code>MS_RDONLY</code>
<code>bind-propagation</code>	<code>rprivate</code> (default), <code>rshared</code> , <code>rslave</code> – cómo se propagan mounts nuevos
<code>:z / :Z</code>	Reetiqueta para SELinux (compartido / privado)

- No hay copia: **es el mismo archivo**. Lo que escribe el container lo ve el host al instante
- **Tapa** lo que la imagen tuviera en ese path
- En macOS y Windows pasa por la VM (VirtioFS): funciona, pero es **notablemente más lento**

tmpfs y memoria compartida

```
docker run --tmpfs /tmp:size=64m,noexec app
docker run --mount type=tmpfs,dst=/tmp,\
tmpfs-size=64m app
```

Un filesystem en **RAM** dentro del mount namespace.
Se destruye con el container y **nunca toca el disco**.

Cuenta contra el **límite de memoria del cgroup**.

Para qué

- Material sensible que no debe persistir (es donde Compose monta los secrets)
- Scratch de alta frecuencia: caches, archivos temporales
- Complemento de `--read-only`: filesystem inmutable + `/tmp` escribible

`--shm-size` ajusta `/dev/shm` (default 64 MB), que suele quedar chico para Chrome o PostgreSQL.

Drivers de volumen

El driver `local` es el default, pero acepta **opciones de mount**: podés montar NFS o CIFS sin plugins.

Los plugins de terceros (NFS, Ceph, EBS, Portworx) exponen almacenamiento remoto como un volumen normal.

En Kubernetes esta idea se estandariza como **CSI**.

```
docker volume create --driver local \  
  --opt type=nfs \  
  --opt o=addr=10.0.0.5,rw \  
  --opt device=:/exports/datos \  
  nfs-datos
```

```
# bind con opciones, como volumen nombrado  
docker volume create --driver local \  
  --opt type=nfs --opt o=bind \  
  --opt device=/srv/datos datos
```

Comandos y respaldo

```
docker volume create datos      docker volume ls -f dangling=true
docker volume inspect datos     docker volume rm datos
```

```
# backup: un container efímero que ve el volumen y un directorio del host
```

```
docker run --rm -v datos:/data:ro -v "$PWD":/backup alpine \
    tar czf /backup/datos.tar.gz -C /data .
```

```
# restore
```

```
docker run --rm -v datos:/data -v "$PWD":/backup alpine \
    tar xzf /backup/datos.tar.gz -C /data
```

`docker volume prune` borra los volúmenes que ningún container usa: **con el stack bajado, eso incluye tus datos.**

Permisos: el problema de los UID

El kernel solo compara **números**. El usuario node (UID 1000) de la imagen y tu usuario del host son el mismo UID o no lo son; los nombres no existen del lado del kernel.

Síntoma: `permission denied` sobre un directorio bind-montado que se ve perfectamente accesible.

```
docker run -u "$(id -u):$(id -g)" -v "$PWD":/app node ...
```

- Volumen **vacío**: hereda el dueño del path en la imagen
- Bind mount: manda **el dueño del host**, sin traducción
- Archivos creados por un container que corre como root quedan **de root en tu host**

El **user namespace** (`users-remap`) mapea los UID del container a un rango sin privilegios del host.

Cuándo usar cada tipo

Necesito	Uso
Datos de una base que deben sobrevivir	Volumen nombrado
Editar código y ver el cambio sin rebuildear	Bind mount (solo desarrollo)
Un archivo de configuración del host	Bind mount : ro
Secretos o temporales que no deben tocar disco	tmpfs
Compartir datos entre containers	Volumen nombrado
Almacenamiento de red	Volumen con driver/opciones

Regla: **volumen** si el dato importa · **bind** si el path del host importa · **tmpfs** si no debe persistir.

Quiz rápido

1. ¿Por qué dos containers pueden escuchar en el puerto 80 al mismo tiempo?
2. ¿Qué diferencia práctica hay entre la red bridge por defecto y una creada por vos?
3. `docker run -p 8080:80`: ¿qué objeto del kernel se crea y en qué tabla?
4. Borro el container de la base con `docker rm`. ¿Perdí los datos?
5. La app anda con `docker exec` pero no desde el host. ¿Primera hipótesis?

Resumen

- El docker CLI es un **cliente de la API** del daemon; el socket es acceso root
- Cada flag de run se traduce en **objetos del kernel**: netns, cgroups, mounts, iptables
- **Red**: bridge propio como default sensato; publicar un puerto es **DNAT**, y el firewall del host no alcanza
- **Volúmenes**: la capa de escritura es efímera – volumen si el dato importa, bind si el path importa, tmpfs si no debe persistir
- Todo esto es lo que en la próxima clase se escribe **declarativamente** en Compose

Para seguir leyendo

- **Docker CLI reference** – docs.docker.com/reference/cli/docker
- **Networking overview** – docs.docker.com/engine/network
- **Packet filtering y firewalls** – docs.docker.com/engine/network/packet-filtering-firewalls
- **Storage: volumes, binds, tmpfs** – docs.docker.com/engine/storage
- **netshoot** – github.com/nicolaka/netshoot

¿Preguntas?